

Unity를 이용한 2D 게임프로그래밍

동명대학교 게임학부

Lecture 10

Unity로 체험하는 C#의 핵심 기능들

C# Feature Lab □ 작은 실험들의 모음

강영민

동명대학교 게임학부

대상 독자

이미 Unity 2D 게임 개발의 기초(MonoBehaviour, Update, Inspector, 컴포넌트 등)는 알고 있는 학생. C#의 핵심 기능 6가지를 “**Unity**라는 무대 위에서 작은 실험으로 직접 만져 보는” 것이 목표이다.

실습 환경: Unity Editor + Visual Studio Code (또는 Visual Studio)

Contents

1 강의 개요	2
1.1 이 강의의 주인공은 “C#”이다	2
1.2 학습 목표	2
1.3 왜 “Lab(실험실)” 형식인가?	2
1.4 이전 강의들과의 연결	2
2 프로젝트 준비	3
2.1 Unity 프로젝트 만들기	3
2.2 공통 도우미 스크립트: LogPanel.cs (선택)	3
3 Lab 1 — enum과 switch 표현식	3
3.1 문제	4
3.2 콘솔에서라면	4
3.3 Unity 데모	4
3.4 Unity 구현 가이드	5
3.5 여기서 배우는 C# 개념	6
4 Lab 2 — 프로퍼티와 캡슐화	7
4.1 문제	7
4.2 콘솔에서라면	7
4.3 Unity 데모	8
4.4 Unity 구현 가이드	8
4.5 여기서 배우는 C# 개념	9
5 Lab 3 — 인터페이스 (interface)	10
5.1 문제	10
5.2 콘솔에서라면	10
5.3 Unity 데모	11
5.4 Unity 구현 가이드	12
5.5 여기서 배우는 C# 개념	13
6 Lab 4 — delegate, Action, event	14
6.1 문제	14
6.2 콘솔에서라면	14
6.3 Unity 데모	15
6.4 Unity 구현 가이드	16
6.5 여기서 배우는 C# 개념	17
7 Lab 5 — 제네릭 List<T>와 LINQ 맛보기	18
7.1 문제	18
7.2 콘솔에서라면	18
7.3 Unity 데모	19
7.4 Unity 구현 가이드	20
7.5 여기서 배우는 C# 개념	20
8 Lab 6 — Coroutine과 람다	22
8.1 문제	22
8.2 콘솔이라면 비슷하게	22
8.3 Unity 데모	22
8.4 Unity 구현 가이드	23

8.5	여기서 배우는 C# 개념	24
9	이번 강의에서 배운 C# 개념 정리	25
9.1	한눈에 보기	25
9.2	Unity는 우리에게 무엇을 해 주었는가?	25
10	단계별 도전 과제	25
10.1	Phase 1 — 같은 Lab에 기능 한 줄 더	26
10.2	Phase 2 — 두 Lab을 합치기	26
10.3	Phase 3 — 자유 실험	26
11	이번 강의에서 배운 핵심 개념	27

1. 강의 개요

1.1. 이 강의의 주인공은 “C#”이다

이 과목은 **Unity를 이용한 2D 게임프로그래밍** 강의다. 그동안 우리는 Unity 위에서 2D 게임을 만들어 왔다. 이번 강의에서는 게임을 만드는 것이 아니라, 게임 코드의 뼈대가 되는 **C# 기능 자체를 Unity라는 무대 위에서 직관적으로 체험해 본다.**

- 콘솔: `Console.WriteLine("HP: " + hp);` → 글자만 보인다.
- Unity: `cube.color = Color.red;` → 큐브가 실제로 빨개진다.

같은 `int`, 같은 `enum`, 같은 `event`라도, 화면 위 객체의 색이 바뀌고 위치가 움직이는 모습으로 보면 머리에 더 잘 새겨진다. 이번 강의는 **C#의 핵심 기능 6가지**를 **Unity라는 무대 위에서 작은 실험으로 직접** 만져 보는 시간이다.

1.2. 학습 목표

- C#의 `enum`, 프로퍼티, `interface`, `event/delegate`, 제네릭/LINQ, `Coroutine`/람다 — 이 6가지 기능이 어떤 문제를 해결하기 위해 존재하는지 이해한다.
- 각 기능을 콘솔이 아닌 **Unity 화면**에서 직접 동작시켜 본다.
- 기능마다 작은 **Lab(실험실)** 스크립트 하나를 작성하고, 키 입력으로 동작을 체험한다.
- 게임을 “완성”하는 것이 목표가 아니라, C# 기능 하나하나를 **확실히 손에 익히는 것**이 목표다.

1.3. 왜 “Lab(실험실)” 형식인가?

게임을 한 편 만드는 것도 좋지만, 그러면 “이 코드 한 줄이 정말로 무슨 일을 하는가”가 게임 전체의 흐름에 묻힌다. 이번 강의는 그 반대다. 큰 게임을 만들지 않는다. 대신 **6개의 작은 실험**을 만든다.

Lab의 형식

1. 문제: “이런 상황이라면 어떻게 코드를 써야 할까?”
2. 콘솔 비교: 일반 C# 콘솔 프로그램이라면 어떻게 했을까?
3. Unity 데모: 작은 `GameObject` + 짧은 스크립트.
4. 키 입력: 1, 2, H, D, A, SPACE 등으로 동작 확인.
5. 실험해 보기: 코드의 한 줄을 바꾸면 결과가 어떻게 달라지는지 손으로 확인.

1.4. 이전 강의들과의 연결

강의	무엇을 배웠나	이번 강의에서의 역할
Lec 1-8	Unity 2D 게임 개발 경험	이번 Lab에서 “쓰는 도구”
Lec 9	Unity C# 스크립트 기초	<code>MonoBehaviour</code> , <code>Update</code> 는 안다고 가정
Lec 10	C# 핵심 기능 6가지를 시각적 실험으로	언어 기능을 손에 익히기

2. 프로젝트 준비

2.1. Unity 프로젝트 만들기

Unity Hub에서 **New Project** → **3D (Built-in Render Pipeline)** 또는 **2D** 어느 쪽도 무방하다 (이번 강의는 3D 큐브를 가정). 프로젝트 이름은 CSharpFeatureLab.

Lab 공통 셋업

썬에 다음 GameObject들을 만들어 둔다 (각 Lab에서 재사용).

- Main Camera — 기본값 그대로.
- Directional Light — 기본값 그대로 (3D 모드라면).
- LabCube — 3D Object → Cube. 위치 (0, 0, 0). Lab 1에서 “무드 큐브”로 사용한다.
- LabRoot — 빈 GameObject. 우리가 작성할 Lab 스크립트들을 여기에 붙인다.
- Canvas + LogText — UI Canvas 안에 TextMeshPro - Text 하나. LogText라 이름 두고 화면 좌상단에 배치. 결과를 화면에서도 확인하기 위함.

콘솔 창(Window → General → Console)을 열어 두고 Debug.Log 결과를 동시에 본다.

2.2. 공통 도우미 스크립트: LogPanel.cs (선택)

Debug.Log 만 써도 충분하지만, 화면에서 결과를 직접 확인하고 싶다면 다음 스크립트를 Canvas에 붙인다.

```

1 using TMPro;
2 using UnityEngine;
3
4 public class LogPanel : MonoBehaviour
5 {
6     public static LogPanel Instance { get; private set; }
7     [SerializeField] private TMP_Text text;
8
9     private string buffer = "";
10
11     void Awake() { Instance = this; }
12
13     public void Print(string msg)
14     {
15         Debug.Log(msg);           // Console
16         buffer = msg + "\n" + buffer; // newest on top
17         if (buffer.Length > 800)
18             buffer = buffer.Substring(0, 800);
19         if (text != null) text.text = buffer;
20     }
21 }
```

이후 Lab 코드에서는 Debug.Log(...) 대신 LogPanel.Instance.Print(...)을 사용해도 된다. 이 강의에서는 둘 다 허용한다.

3. Lab 1 □ enum과 switch 표현식

3.1. 문제

세 가지 “기분(mood)” 상태 중 하나를 표현하고 싶다. int 0/1/2로 쓰면 “state == 1이 무슨 뜻이지?”라는 혼란이 생기고, 오타도 잡히지 않는다. 이름 있는 상수 집합이 필요하다.

3.2. 콘솔에서라면

일반 C# 콘솔 프로그램이었다면 이렇게 했을 것이다.

```

1 public enum Mood { Happy, Sad, Angry }
2
3 class Program
4 {
5     static void Main()
6     {
7         Mood m = Mood.Happy;
8         string msg = m switch
9         {
10             Mood.Happy => "I am happy",
11             Mood.Sad   => "I am sad",
12             Mood.Angry => "I am angry",
13             -          => "???"
14         };
15         Console.WriteLine(msg);
16     }
17 }
```

콘솔이라면 “I am happy”라는 글자가 한 번 출력되고 끝이다.

3.3. Unity 데모

큐브 하나를 두고, 키 1/2/3을 누르면 Mood가 바뀌면서 큐브의 색이 즉시 바뀐다.

```

1 using UnityEngine;
2
3 public enum Mood { Happy, Sad, Angry }
4
5 public class Lab1_Enum : MonoBehaviour
6 {
7     [SerializeField] private Renderer targetRenderer;
8     private Mood currentMood = Mood.Happy;
9
10    void Start()
11    {
12        Apply(currentMood);
13    }
14
15    void Update()
16    {
17        if (Input.GetKeyDown(KeyCode.Alpha1)) Apply(Mood.Happy);
18        if (Input.GetKeyDown(KeyCode.Alpha2)) Apply(Mood.Sad);
19        if (Input.GetKeyDown(KeyCode.Alpha3)) Apply(Mood.Angry);
20    }
21
22    void Apply(Mood m)
23    {
24        currentMood = m;
25
26        // C# 8+ switch expression
```

```

27     Color color = m switch
28     {
29         Mood.Happy => Color.yellow,
30         Mood.Sad   => new Color(0.3f, 0.5f, 1f),
31         Mood.Angry => Color.red,
32         -          => Color.white
33     };
34
35     targetRenderer.material.color = color;
36     Debug.Log($"Mood = {m}");
37 }
38 }

```

3.4. Unity 구현 가이드

1. 씬에 큐브 만들기

Hierarchy 창에서 우클릭 → 3D Object → Cube. 새로 생긴 객체의 이름을 MoodCube로 바꾼다. Inspector에서 Transform.Position을 (0, 0, 0)으로 둔다.

2. Lab1 GameObject 만들기

Hierarchy 창에서 우클릭 → Create Empty. 이름을 Lab1로 바꾼다. 이 빈 객체에 우리 스크립트를 붙일 것이다.

3. 스크립트 작성

Project 창에서 Assets/Scripts/ 폴더 우클릭 → Create → C# Script. 이름을 Lab1_Enum으로 만든다. 더블 클릭해 VS Code에서 열고, 위의 Lab1_Enum.cs 코드를 복사해 넣는다.

4. 스크립트 부착

Project 창의 Lab1_Enum 스크립트를 드래그해 Hierarchy의 Lab1 객체 위에 놓는다. Lab1을 선택하면 Inspector에 Lab1_Enum (Script) 컴포넌트가 보인다.

5. Inspector 슬롯 연결

Target Renderer라는 슬롯이 비어 있다 (None (Renderer)). Hierarchy의 MoodCube를 드래그해 이 슬롯에 떨어뜨린다.

6. 재생

화면 상단의 Play 버튼을 누른다. 키 1, 2, 3을 누를 때마다 큐브가 노란/파란/빨간색으로 바뀐다. Console 창(Window → General → Console)에 Mood = ...가 함께 출력된다.

드래그한 것은 GameObject인데, 왜 Renderer 슬롯에 들어가나?

Inspector 슬롯의 타입이 Renderer일 때 GameObject를 드래그하면, Unity가 그 GameObject에 붙어 있는 컴포넌트들 중 타입이 일치하는 것을 자동으로 찾아 슬롯에 채운다. MoodCube에는 MeshRenderer(=Renderer의 자식 클래스)가 있으므로 그게 들어간다. 이는 Unity Editor의 편의 기능이며, 코드로는 moodCube.GetComponent<Renderer>()와 같다.

3.5. 여기서 배우는 C# 개념

enum: 이름 있는 상수의 집합

enum Mood { Happy, Sad, Angry }는 내부적으로 0, 1, 2라는 int일 뿐이다. 하지만 코드에 `state == 1`이라고 쓰는 것과 `state == Mood.Sad`라고 쓰는 것은 읽기 쉬움이 천지차이다. 오타는 컴파일러가 잡아 주고, IDE는 자동 완성을 도와준다.

switch 표현식 (C# 8.0+)

전통적인 switch는 문(statement)이라 break와 case를 일일이 써야 했다. C# 8부터 switch는 식(expression)이 되어 값을 반환한다.

```

1 // Old style (statement)
2 Color color;
3 switch (m)
4 {
5     case Mood.Happy: color = Color.yellow; break;
6     case Mood.Sad:   color = Color.blue;   break;
7     default:        color = Color.white;  break;
8 }
9
10 // New style (expression)
11 Color color = m switch
12 {
13     Mood.Happy => Color.yellow,
14     Mood.Sad   => Color.blue,
15     _          => Color.white
16 };

```

짧고, “무엇을 얻는가”가 한눈에 보인다. _는 “나머지 모든 경우”.

[SerializeField] private이 무슨 뜻인가?

private 필드는 같은 클래스 안에서만 접근 가능하다 (캡슐화). 하지만 그렇게만 두면 Unity Inspector에서도 보이지 않아 값을 조정할 수 없다. [SerializeField]는 “이 필드를 Inspector에 보여 줘”라고 Unity에 알려 주는 어트리뷰트(Attribute)다.

결과적으로 [SerializeField] private float speed = 8f;는 다음을 의미한다:

- 다른 C# 클래스에서는 lab.speed로 접근 불가.
- Unity Editor의 Inspector에서는 값을 직접 수정 가능.

대괄호 안의 [Foo] 같은 표기는 모두 어트리뷰트이며, “클래스/필드/메서드에 꼬리표를 붙인다”고 이해하면 된다.

\$"...{x}..." □ 문자열 보간(string interpolation)

큰따옴표 앞에 \$를 붙이면 문자열 안의 {식}이 그 식의 값으로 치환된다.

```

1 int n = 42;
2 string s = $"n = {n}";           // "n = 42"
3 string t = $"sum = {a + b}";     // expressions also work
4 string u = $"avg = {x:F2}";      // F2 = two decimal places

```

콜론 뒤의 “F2”, “D6” 등은 형식 지정자. F2는 “소수점 둘째 자리까지”, D6은 “정수 6자리, 빈 자리는 0”.

옛 방식의 `string.Format("n = {0}", n)`이나 `"n = " + n`보다 훨씬 읽기 쉽다.

실험해 보기

- Mood에 Excited를 추가하고, 키 4에 연결해 보자. switch의 _ 가지가 어떤 역할인지 확인.
- Mood.Happy를 정수로 바꾸려면? `(int)Mood.Happy`를 `Debug.Log`로 출력해 보자.
- `enum Mood : byte`로 하면 어떻게 달라질까? (저장 크기 변화)

4. Lab 2 □ 프로퍼티와 캡슐화

4.1. 문제

캐릭터의 HP는 외부에서 읽는 것은 OK, 마음대로 쓰는 것은 안됨이어야 한다. 누구나 `character.hp = 99999`로 만들 수 있으면 게임 규칙이 무너진다.

4.2. 콘솔에서라면

```

1 class Character
2 {
3     public int hp;    // anyone can write to this - dangerous
4 }
5
6 class Program
7 {
8     static void Main()
9     {
10         var c = new Character();
11         c.hp = 100;
12         c.hp = -50;    // negative? no way to block this
13     }
14 }
```

이 코드는 캡슐화 위반이다. 프로퍼티로 다음과 같이 보호한다.

```

1 class Character
2 {
3     public int Hp { get; private set; }    // public read, blocked write
4     private int maxHp = 10;
5
6     public Character() { Hp = maxHp; }
7
8     public void Damage(int amount)
9     {
10         Hp = Math.Max(0, Hp - amount);
11     }
12
13     public void Heal(int amount)
14     {
15         Hp = Math.Min(maxHp, Hp + amount);
16     }
17 }
```

4.3. Unity 데모

같은 Character 로직을 키 입력에 연결한다. D 키로 데미지, H 키로 회복, R 키로 “밖에서 직접 쓰기 시도”(컴파일 에러).

```

1 using UnityEngine;
2
3 public class Lab2_Property : MonoBehaviour
4 {
5     [SerializeField] private int maxHp = 10;
6     public int Hp { get; private set; }
7
8     void Start()
9     {
10         Hp = maxHp;
11         Report();
12     }
13
14     void Update()
15     {
16         if (Input.GetKeyDown(KeyCode.D)) { Damage(1); Report(); }
17         if (Input.GetKeyDown(KeyCode.H)) { Heal(1); Report(); }
18
19         // The line below would NOT compile, by design:
20         // if (Input.GetKeyDown(KeyCode.R)) Hp = 9999;
21     }
22
23     public void Damage(int amount) => Hp = Mathf.Max(0, Hp - amount);
24     public void Heal(int amount) => Hp = Mathf.Min(maxHp, Hp + amount);
25
26     void Report()
27     {
28         Debug.Log($"HP = {Hp}/{maxHp}");
29     }
30 }

```

4.4. Unity 구현 가이드

1. Lab2 GameObject 만들기

Hierarchy에서 우클릭 → Create Empty → 이름 Lab2. 이번 Lab은 시각적 객체가 필요 없다 (Console로만 결과 확인).

2. 스크립트 작성 & 부착

Assets/Scripts/Lab2_Property.cs를 만들어 코드 입력 후 Lab2 객체에 드래그.

3. Inspector에서 값 확인

Lab2를 선택해 Inspector를 보면 Max Hp = 10이 보인다. 이는 코드의 [SerializeField] private int maxHp = 10; 덕분. Hp는 보이지 않는다 — [SerializeField]가 없는 그냥 자동 프로퍼티이기 때문. 값을 20으로 바꾸면 시작 HP가 20/20이 된다.

4. Console 창 열기

Window → General → Console. Play 직전에 Console을 봐 두면 Debug.Log 출력이 즉시 보인다.

5. 재생

Play → D 키로 데미지, H 키로 회복. Console에 HP = 9/10, HP = 8/10 같은 로그가 누적된

다. Hp가 0 미만 또는 maxHp 초과로 가지 않는다 (Mathf.Max/Mathf.Min 덕분).

4.5. 여기서 배우는 C# 개념

프로퍼티: { get; private set; }

필드(field)는 “값을 담는 변수”. 프로퍼티(property)는 “필드처럼 보이지만 실은 메서드 두 개”다.

- `public int Hp { get; set; }` — 외부 읽기 OK, 외부 쓰기 OK (필드와 거의 같음).
- `public int Hp { get; private set; }` — 외부 읽기 OK, 외부 쓰기 차단.
- `public int Hp { get; }` — 읽기 전용 (생성자에서만 초기화 가능).

이 한 줄로 캡슐화의 의도가 분명히 드러난다. “Hp는 외부에서 보는 값이다”.

식 본문 메서드: =>

`Damage(int amount) { Hp = ...; }`처럼 한 줄짜리 메서드는 =>로 줄일 수 있다.

```
1 public void Damage(int amount) => Hp = Mathf.Max(0, Hp - amount);
```

읽기 좋고 짧다. 본문이 두 줄 이상이면 사용하지 않는다.

{ get; private set; }을 풀어 보면?

`public int Hp { get; private set; }`는 자동 구현 프로퍼티(auto-implemented property)다. 컴파일러가 다음과 같은 코드를 자동으로 만들어 준다.

```
1 private int _hp;           // hidden backing field generated by the compiler
2 public int Hp
3 {
4     get { return _hp; }     // public read for anyone
5     private set { _hp = value; } // write only inside this class
6 }
```

value는 set이 받은 값을 가리키는 약속된 키워드. `Hp = 10;`이라고 쓰면 set의 value에 10이 들어간다.

읽기/쓰기 옵션을 다양하게 줄 수 있다:

- { get; set; } — 외부 읽기 OK, 외부 쓰기 OK
- { get; private set; } — 외부 읽기 OK, 외부 쓰기 차단
- { get; } — 읽기만, 쓰기 불가능 (생성자에서만 초기화)

=> 화살표의 세 가지 얼굴

초보자에게 헷갈리는 점: C#의 =>는 문맥에 따라 세 가지로 쓰인다. 모두 “왼쪽이 들어가서 오른쪽이 나온다”는 비슷한 느낌이지만, 자리와 의미가 다르다.

(1) 식 본문 멤버 (Lab 2, Lab 3에서 만남) “이 메서드/프로퍼티는 이 식 한 줄을 반환한다”.

```
1 public void Damage(int n) => Hp -= n;           // method
2 public string Name => "Circle";                 // property
```

```
3 public float Area() => Mathf.PI * r * r;    // method
```

같은 자리에 { ... }를 쓸 수도 있다. =>는 단축 표기일 뿐.

(2) 람다 식 (Lab 5, Lab 6에서 만남) “이 매개변수들을 받아서 이 식을 반환하는 함수”.

```
1 x => x >= 80                // takes int, returns bool   (Func<int, bool>)
2 n => Debug.Log($"Tick {n}")  // takes int, returns void  (Action<int>)
3 () => Debug.Log("hi")        // no parameters           (Action)
4 (a, b) => a + b              // two parameters, returns sum (Func<int,int,int>)
```

이는 이름 없는 함수이며, 변수에 담거나 메서드 인자로 넘길 수 있다.

(3) switch 식의 가지 (Lab 1에서 만남) “이 패턴이면 이 값”.

```
1 Color c = m switch {
2     Mood.Happy => Color.yellow,    // when Happy, return yellow
3     Mood.Sad   => Color.blue,
4     -          => Color.white
5 };
```

기억하는 법: 어디에 있느냐를 보면 된다. 메서드/프로퍼티 선언 자리면 (1), 인자/변수 자리에서 함수 자체를 만드는 거면 (2), switch 안이면 (3).

실험해 보기

- 위 코드의 `public int Hp { get; private set; }`을 `public int Hp;`으로 바꿔 보자. 그러면 다른 스크립트(예: Lab2_Bad.cs)에서 `lab.Hp = 9999`로 마음대로 바꿀 수 있다. “프로퍼티”의 의의가 체감된다.
- `Hp { get; private set; }`을 `Hp { get; }`으로 바꿔 보자. 이제 Damage 안의 `Hp = ...`도 컴파일 에러가 난다. 어디까지 “쓰기”를 허용할 것인가를 한 줄로 조절하는 감각을 익힌다.

5. Lab 3 □ 인터페이스 (interface)

5.1. 문제

여러 도형(Circle, Rectangle, Triangle)의 면적을 구하고 출력하고 싶다. 도형마다 면적을 구하는 공식은 다르지만, “면적을 구한다”라는 능력은 공통이다. 이 “공통 능력”만 약속해 두면, 도형이 늘어나도 호출 코드는 바뀌지 않는다.

5.2. 콘솔에서라면

```
1 public interface IShape
2 {
3     string Name { get; }
4     float Area();
5 }
6
7 public class Circle : IShape
8 {
9     public float Radius;
10    public Circle(float r) { Radius = r; }
11    public string Name => "Circle";
12    public float Area() => Mathf.PI * Radius * Radius;
```

```

13 }
14
15 public class Rectangle : IShape
16 {
17     public float W, H;
18     public Rectangle(float w, float h) { W = w; H = h; }
19     public string Name => "Rectangle";
20     public float Area() => W * H;
21 }
22
23 class Program
24 {
25     static void Main()
26     {
27         IShape[] shapes = { new Circle(2f), new Rectangle(3f, 4f) };
28         foreach (var s in shapes)
29             Console.WriteLine($"{s.Name}: {s.Area():F2}");
30     }
31 }

```

5.3. Unity 데모

같은 IShape를 Unity에서 사용한다. A 키를 누르면 모든 도형의 면적이 콘솔/화면에 출력된다.

```

1 using UnityEngine;
2
3 public interface IShape
4 {
5     string Name { get; }
6     float Area();
7 }
8
9 public class CircleShape : IShape
10 {
11     private float radius;
12     public CircleShape(float r) { radius = r; }
13     public string Name => "Circle";
14     public float Area() => Mathf.PI * radius * radius;
15 }
16
17 public class RectangleShape : IShape
18 {
19     private float w, h;
20     public RectangleShape(float w, float h) { this.w = w; this.h = h; }
21     public string Name => "Rectangle";
22     public float Area() => w * h;
23 }
24
25 public class Lab3_Interface : MonoBehaviour
26 {
27     private IShape[] shapes;
28
29     void Start()
30     {
31         shapes = new IShape[]
32         {
33             new CircleShape(2f),
34             new RectangleShape(3f, 4f),

```

```

35         new CircleShape(1.5f),
36     };
37 }
38
39 void Update()
40 {
41     if (Input.GetKeyDown(KeyCode.A))
42     {
43         float total = 0f;
44         foreach (var s in shapes)
45         {
46             Debug.Log($"{s.Name}: area = {s.Area():F2}");
47             total += s.Area();
48         }
49         Debug.Log($"Total area = {total:F2}");
50     }
51 }
52 }

```

5.4. Unity 구현 가이드

1. Lab3 GameObject 만들기

Hierarchy → Create Empty → 이름 Lab3.

2. 스크립트 파일 정리

이번 Lab에는 클래스가 여러 개 등장한다 (IShape, CircleShape, RectangleShape, Lab3_Interface). 한 .cs 파일에 모두 넣어도 되지만, 권장 구조는 한 파일 = 한 클래스. Assets/Scripts/ 안에 다음 네 파일을 만든다:

- IShape.cs — 인터페이스 정의
- CircleShape.cs — 원
- RectangleShape.cs — 사각형
- Lab3_Interface.cs — MonoBehaviour

3. 스크립트 부착

Lab3_Interface만 Lab3 GameObject에 드래그. IShape, CircleShape, RectangleShape는 붙이지 않는다. 이들은 MonoBehaviour가 아니라 보통 C# 클래스이며, Lab3_Interface.Start() 안에서 new로 만들어진다.

4. Console 창 열기

Window → General → Console.

5. 재생

Play → A 키. Console에 다음과 같은 로그가 출력된다:

```

Circle: area = 12.57
Rectangle: area = 12.00
Circle: area = 7.07
Total area = 31.63

```

Unity의 “파일명 = 클래스명” 규칙

MonoBehaviour를 상속한 클래스는 파일 이름과 클래스 이름이 반드시 같아야 Unity가 컴포넌트로 인식한다. 하지만 보통 **C#** 클래스나 인터페이스는 그 규칙이 없다. 한 파일에 여러 개의 비-MonoBehaviour 클래스를 뒤도 동작하지만, 읽기 좋고 찾기 쉽도록 “한 파일 한 클래스”를 권한다 (Lec 9에서도 다룬 규칙).

5.5. 여기서 배우는 C# 개념

interface: “무엇을 할 수 있는가” 약속

인터페이스는 메서드/프로퍼티의 목록만 정의한다. 구현은 각 클래스가 자유롭게 한다.

- 클래스는 여러 인터페이스를 동시에 구현할 수 있다 (다중 “상속”의 안전한 형태).
- 인터페이스 자체는 **new**로 만들 수 없다 — 구현체가 있어야.
- 이름은 보통 **I**로 시작 (IShape, IComparable, IEnumerable...).

상속(is-a) vs 인터페이스(can-do) 상속은 “Cat은 Animal이다”라는 **is-a** 관계.
인터페이스는 “Cat은 울 수 있다”라는 **can-do** 관계.
“면적을 구할 수 있다”는 후자에 해당하므로 인터페이스가 적합하다.

프로퍼티가 인터페이스에 들어갈 수 있다

이번 IShape는 Name이라는 프로퍼티도 약속에 포함시켰다. 인터페이스는 메서드뿐 아니라 프로퍼티/이벤트/인덱서도 약속할 수 있다.

public string Name => "Circle";이 무슨 뜻인가?

식 본문 프로퍼티(expression-bodied property) — “이 프로퍼티는 항상 이 식의 값을 반환한다”.

풀어 쓰면 다음과 같다:

```
1 // Short form
2 public string Name => "Circle";
3
4 // Same meaning, expanded
5 public string Name
6 {
7     get { return "Circle"; }
8 }
```

세 가지 사실:

- **set**이 없으므로 외부에서 쓸 수 없다 (circle.Name = "X" → 컴파일 에러).
- 호출할 때마다 식의 값을 **매번** 평가해서 반환한다 (이번 경우 항상 "Circle").
- 메서드(Name())가 아니라 프로퍼티이므로 사용 시 **괄호 없이** circle.Name이라고 쓴다.

이는 “=>의 세 얼굴” 중 (1) 식 본문 멤버에 해당한다 (Lab 2 박스 참고).

foreach와 var

foreach (var s in shapes)는 “shapes의 원소를 하나씩 꺼내 s에 담아 본다”라는 뜻.

var: 타입 추론 var s는 “타입을 컴파일러가 알아서 결정해 주세요”라는 표시. shapes가 IShape[]이므로 s의 타입은 IShape로 자동 결정된다.

주의: var는 “아무 타입이나” 의미가 아니다. 타입은 컴파일 시점에 결정되며, 한 번 정해지면 바꿀 수 없다. JavaScript의 var나 Python의 동적 타입과는 전혀 다르다.

실험해 보기

- TriangleShape를 추가하고 shapes 배열에 끼워 넣자.
Lab3_Interface.cs의 Update는 한 줄도 고치지 않아도 새 도형이 같이 출력된다. “인터페이스의 효과”를 손으로 느낀다.
- IShape에 Perimeter()를 추가해 보자. 그러면 모든 구현 클래스가 컴파일 에러를 낸다.
약속을 안 지키면 컴파일러가 막아 준다는 점이 중요하다.

6. Lab 4 □ delegate, Action, event

6.1. 문제

“숫자 변수가 바뀌면 여러 곳에 알리고 싶다.” UI 텍스트도 갱신되어야 하고, 사운드도 재생되어야 하고, 화면 흔들림도 일어나야 한다. 이를 변수 쪽에서 직접 호출하면, 알림 받는 쪽이 새로 추가될 때마다 코드가 늘어난다.

6.2. 콘솔에서라면

이벤트로 “소문”을 내고, 관심 있는 사람만 듣는 구조를 만들 수 있다.

```

1 class Counter
2 {
3     public event Action<int> OnChanged;    // notification only
4     private int n = 0;
5     public void Increment()
6     {
7         n++;
8         OnChanged?.Invoke(n);
9     }
10 }
11
12 class Program
13 {
14     static void Main()
15     {
16         var c = new Counter();
17         c.OnChanged += v => Console.WriteLine($"Logger: {v}");
18         c.OnChanged += v => Console.WriteLine($"Beeper: beep {v} times!");
19
20         c.Increment();    // triggers BOTH subscribers
21         c.Increment();
22     }
23 }
```


6.3. Unity 데모

스페이스 키를 누르면 Counter가 1씩 증가하고, 여러 구독자가 자동으로 반응한다.

```

1 using System;
2 using UnityEngine;
3
4 public class Lab4_Counter : MonoBehaviour
5 {
6     public event Action<int> OnChanged;
7     private int value_ = 0;
8
9     void Update()
10    {
11        if (Input.GetKeyDown(KeyCode.Space))
12        {
13            value_++;
14            OnChanged?.Invoke(value_);
15        }
16    }
17 }

```

구독자(observer)는 별도 컴포넌트로 만든다.

```

1 using UnityEngine;
2
3 public class Lab4_Logger : MonoBehaviour
4 {
5     [SerializeField] private Lab4_Counter source;
6
7     void OnEnable() { source.OnChanged += React; }
8     void OnDisable() { source.OnChanged -= React; }
9
10    void React(int v) => Debug.Log($"Logger heard: {v}");
11 }
12
13 public class Lab4_ColorChanger : MonoBehaviour
14 {
15     [SerializeField] private Lab4_Counter source;
16     [SerializeField] private Renderer target;
17
18     void OnEnable() { source.OnChanged += React; }
19     void OnDisable() { source.OnChanged -= React; }
20
21     void React(int v)
22     {
23         // 0 -> dark, 10 -> bright, then loops
24         float t = (v % 10) / 10f;
25         target.material.color = Color.Lerp(Color.black, Color.cyan, t);
26     }
27 }

```

6.4. Unity 구현 가이드

이번 Lab은 여러 컴포넌트가 서로 참조하므로, 단계가 조금 길다. 차분히 따라가자.

1. 발신자 **GameObject** 만들기

Hierarchy → Create Empty → 이름 Lab4_Source. Lab4_Counter.cs를 작성하여 이 객체에 드래그.

2. 큐브 만들기

Hierarchy → 3D Object → Cube → 이름 Lab4_Cube. 위치 (2, 0, 0) 정도로 두면 다른 Lab의 큐브와 안 겹친다.

3. 구독자 **GameObject** 만들기

Hierarchy → Create Empty → 이름 Lab4_Subscribers. 이 객체에 두 개의 스크립트를 모두 드래그:

- Lab4_Logger.cs
- Lab4_ColorChanger.cs

한 GameObject에 여러 컴포넌트가 붙는다. 이는 Unity에서 흔한 패턴이다.

4. Inspector 슬롯 연결 (가장 중요한 단계)

Lab4_Subscribers를 선택. Inspector에 두 컴포넌트가 모두 보인다.

- Lab4_Logger의 Source 슬롯 ← Lab4_Source 드래그. Unity가 자동으로 Lab4_Counter 컴포넌트를 찾아 채운다.
- Lab4_ColorChanger의 Source 슬롯 ← Lab4_Source 드래그.
- Lab4_ColorChanger의 Target 슬롯 ← Lab4_Cube 드래그.

5. 재생

Play → SPACE 키. 콘솔에 로그가 찍히면서 동시에 큐브 색이 변한다. 두 구독자는 서로의 존재를 모른다.

GameObject 구조를 그림으로

Hierarchy:

Main Camera

Directional Light

Lab4_Source <-- Lab4_Counter (발신자)

Lab4_Subscribers <-- Lab4_Logger, Lab4_ColorChanger (수신자 둘)

Lab4_Cube <-- Renderer (색이 바뀜)

이렇게 발신자/수신자/시각적 객체를 세 개의 GameObject로 책임 분리하면 구조가 더 명확해진다. 같은 GameObject에 모두 붙여도 동작은 하지만, 학습 단계에서는 분리해 두는 편이 좋다.

6.5. 여기서 배우는 C# 개념

delegate, Action, event 한 묶음

delegate: 메서드를 변수처럼 담는 그릇. `public delegate void InHandler(int x);`처럼 직접 정의할 수도 있지만, 보통은 ...

Action<T>: “T를 받고 void를 반환하는 delegate”를 .NET이 미리 만들어 둔 것.

Func<T, R>: “T를 받고 R을 반환하는 delegate”.

event: delegate 위에 “외부에서는 +=와 -=만 가능”이라는 안전 장치를 씌운 것. 외부에서 `c.OnChanged()`로 직접 호출하지는 못한다.

?.Invoke(...): 안전 호출

event는 처음에 null이다. 아무도 구독하지 않았는데 `Invoke`를 부르면 `NullReferenceException`. `?.(null-conditional 연산자)`을 쓰면 “왼쪽이 null이 아닐 때만”만 호출한다.

`OnChanged?.Invoke(value_);` == `if (OnChanged != null) OnChanged(value_);`

OnDisable에서 반드시 -=

`+=`로 구독한 메서드는 `-=`로 해제하지 않으면 그 객체가 파괴된 후에도 이벤트가 그쪽을 호출하려 한다. 이는 메모리 누수와 파괴된 객체에 호출이라는 두 가지 문제를 동시에 만든다. 구독은 `OnEnable`, 해제는 `OnDisable`이 안전한 짝.

Action<T>, Func<T, R>, delegate의 정체

delegate는 “메서드를 담는 변수의 타입”이다. .NET에는 자주 쓰는 두 가지 모양이 미리 만들어져 있다.

```
1 // "delegate that takes T and returns void"
2 public delegate void Action<T>(T arg);
3
4 // "delegate that takes T and returns R"
5 public delegate R Func<T, R>(T arg);
```

이 둘은 우리가 직접 정의한 게 아니라 .NET이 미리 만들어 놓은 단축이다. 덕분에 우리가 `public delegate void MyHandler(int x);`처럼 매번 새로 정의할 필요가 없다.

- `Action` — 매개변수 없음, 반환 `void`.
- `Action<int>` — `int` 받음, 반환 `void`.
- `Action<int, string>` — `int`와 `string` 받음, 반환 `void`.
- `Func<int, bool>` — `int` 받음, `bool` 반환.
- `Func<int, int, int>` — 마지막이 반환, 앞은 매개변수.

?.와 ?.Invoke(...)

?.는 **null-조건 연산자(null-conditional operator)**. “왼쪽이 null이 아닐 때만” 오른쪽을 실행한다.

```
1 string s = null;
2 int len = s.Length;           // NullReferenceException!
3 int? len2 = s?.Length;       // if s is null, len2 is also null. Safe.
```

이벤트와의 결합

event는 처음에 null이다 (아직 아무도 구독하지 않은). 이때 OnChanged()를 직접 호출하면 NullReferenceException. ? Invoke(...)은 객체 "구독자가 한 명이라도 있는"임을 알지만 그냥 메소드.

```
1 // Equivalent two lines
2 OnChanged?.Invoke(value_);
3 if (OnChanged != null) OnChanged(value_);
```

왜 Invoke를 써야 하나?

event는 외부에서 OnChanged()로 직접 호출이 막혀 있다 (안전 장치). 대신 Invoke()라는 메서드가 있어, "이름과 매개변수 형태가 같은 어떤 메서드든 호출" 가능. (참고: [이름과 매개변수 형태가 같은 어떤 메서드든 호출](#))

OnEnable/OnDisable이 호출되는 시점

Lec 9에서 다룬 MonoBehaviour의 생명 주기 메서드 중 두 개:

- **OnEnable** — 컴포넌트가 활성화될 때마다 호출. Play 시작 직후, 또는 Inspector의 체크박스를 켤 때, 또는 gameObject.SetActive(true) 직후.
- **OnDisable** — 컴포넌트가 비활성화될 때마다 호출. 체크박스 끄기, SetActive(false), 객체 파괴 직전.

이벤트 구독을 Start에서 하고 해제를 OnDestroy에서 해도 동작은 한다. 하지만 OnEnable/OnDisable을 쓰면 컴포넌트가 켜졌다 꺼졌다 할 때마다 자동으로 짝이 맞아 더 안전하다.

실험해 보기

- Lab4_ColorChanger의 enabled를 끄면, 큐브 색은 변하지 않지만 로그는 여전히 찍힌다. 구독자가 “빠지더라도” 발신자는 영향받지 않는다는 점을 확인한다.
- 새 구독자 Lab4_Beeper.cs를 추가해 본다. 발신자(Lab4_Counter)는 한 줄도 수정하지 않는다. 이것이 “느슨한 결합”의 효과다.

7. Lab 5 □ 제네릭 List<T>와 LINQ 맛보기

7.1. 문제

여러 점수를 모아 두고, “총합/평균/최고점/80점 이상의 개수”를 알고 싶다. 배열은 크기가 고정되어 “계속 추가”가 어렵다. 크기가 자동으로 늘어나는 컬렉션이 필요하다.

7.2. 콘솔에서라면

```
1 using System.Collections.Generic;
2 using System.Linq;
3
4 class Program
5 {
6     static void Main()
7     {
8         List<int> scores = new List<int>();
9         scores.Add(85); scores.Add(72); scores.Add(91); scores.Add(60);
10    }
```

```

11 Console.WriteLine($"Count : {scores.Count}");
12 Console.WriteLine($"Sum : {scores.Sum()}");
13 Console.WriteLine($"Avg : {scores.Average():F1}");
14 Console.WriteLine($"Max : {scores.Max()}");
15 Console.WriteLine($">=80 : {scores.Where(s => s >= 80).Count()}");
16 }
17 }

```

7.3. Unity 데모

N 키를 누르면 0~99 범위 랜덤 점수를 List에 추가하고, L 키를 누르면 LINQ로 통계를 출력한다.

```

1 using System.Collections.Generic;
2 using System.Linq;
3 using UnityEngine;
4
5 public class Lab5_Generic : MonoBehaviour
6 {
7     private List<int> scores = new List<int>();
8
9     void Update()
10    {
11        if (Input.GetKeyDown(KeyCode.N))
12        {
13            int s = Random.Range(0, 100);
14            scores.Add(s);
15            Debug.Log($"Added: {s}, Count={scores.Count}");
16        }
17
18        if (Input.GetKeyDown(KeyCode.L))
19        {
20            if (scores.Count == 0)
21            {
22                Debug.Log("No scores yet. Press N to add.");
23                return;
24            }
25
26            int sum = scores.Sum();
27            double avg = scores.Average();
28            int max = scores.Max();
29            int top = scores.Where(x => x >= 80).Count();
30
31            Debug.Log($"Sum={sum}, Avg={avg:F1}, Max={max}, >=80: {top}");
32        }
33
34        if (Input.GetKeyDown(KeyCode.K))
35        {
36            scores.Clear();
37            Debug.Log("Cleared.");
38        }
39    }
40 }

```

7.4. Unity 구현 가이드

1. Lab5 GameObject 만들기

Hierarchy → Create Empty → 이름 Lab5.

2. 스크립트 작성 & 부착

Assets/Scripts/Lab5_Generic.cs를 만들어 코드 입력. Lab5 객체에 드래그해 부착.

3. using 지시문 확인

파일 맨 위에 다음 두 줄이 반드시 있어야 한다:

```
using System.Collections.Generic; // List<T>를 위해
using System.Linq;                // .Sum(), .Where() 등을 위해
```

둘 중 하나라도 빠지면 List나 .Sum()이 “컴파일 에러”로 표시된다. VS Code에서 빨간 줄이 보이면 using부터 점검.

4. Console 창 열기

Window → General → Console.

5. 재생

Play 후 다음 키를 눌러 본다:

- **N** — 점수를 하나 추가 (0~99 랜덤)
- **L** — 통계 출력 (Sum, Avg, ≥80 개수)
- **K** — 리스트 비우기

N을 여러 번 누른 후 L을 눌러 평균이 어떻게 변하는지 본다.

7.5. 여기서 배우는 C# 개념

제네릭 <T>: 타입을 변수처럼

List<int>, List<string>, List<Mood> — 같은 List 클래스이지만 **담는 타입이 다르다**.

List<T> 안에서 T는 “무엇이든 들어올 수 있는 자리”다. 컴파일 시점에 결정되며, List<int>.Add("hi")처럼 잘못된 타입은 **컴파일 에러**로 즉시 잡힌다.

이를 **타입 안전성(type safety)**이라 한다. 옛날 ArrayList는 object로 받아 런타임에야 오류가 드러났다. 제네릭은 그 문제를 해결했다.

LINQ: 컬렉션을 “질의”하는 한 줄들

using System.Linq;만 추가하면, 모든 IEnumerable<T>에 마법 같은 메서드가 붙는다.

- .Sum(), .Average(), .Max(), .Min(), .Count()
- .Where(predicate) — 조건으로 거른다
- .Select(transform) — 각 원소를 변환
- .OrderBy(key) — 정렬
- .GroupBy(key) — 그룹화

이 메서드들은 모두 **함수형 스타일**이다. for를 직접 쓰는 것보다 “무엇을 원하는가”가 한눈에 보인다.

람다(=>)와 함께 `scores.Where(x => x >= 80)`에서 `x => x >= 80`은 람다 식. “x를 받아 x >= 80을 반환하는 익명 함수”다. 함수를 변수처럼 인자로 넘긴다. 이 자리는 `Func<int, bool>` 타입을 받는다.

`x => x >= 80`이 `Func<int, bool>`이 되는 과정

이 짧은 식 한 줄에 많은 일이 일어나고 있다. 풀어 보자.

1. **Where**는 무엇을 받나? `List<int>`의 **Where**는 다음 시그니처를 가진 메서드다:

```
1 IEnumerable<int> Where(Func<int, bool> predicate);
```

즉 “int를 받아 bool을 반환하는 함수”를 인자로 요구한다.

2. 우리가 줄 수 있는 것 다음 두 가지 모두 `Func<int, bool>`을 만든다:

```
1 // Old way: define a separate method, pass as method group
2 bool IsHigh(int x) { return x >= 80; }
3 scores.Where(IsHigh);
4
5 // New way: write the lambda directly as an argument
6 scores.Where(x => x >= 80);
```

람다 `x => x >= 80`은 “x를 받아 x >= 80을 반환하는 함수”. 컴파일러는 “Where가 `Func<int, bool>`을 원한다”는 것을 알고, 람다의 x를 int로, 반환 타입을 bool로 자동 추론한다.

```
1 x => x + 1           // same as (int x) => x + 1
2 () => 42             // no parameters
3 (a, b) => a + b      // two parameters
4 x => { Debug.Log(x); } // block body (multiple lines)
```

매개변수가 하나일 때만 괄호를 생략할 수 있다.

`using System.Linq;`이 하는 일 □ 확장 메서드

`List<int>` 클래스의 코드를 직접 들여다보면 `Sum`, `Where`, `Max` 같은 메서드는 없다. 그런데 `using System.Linq;`만 써 주면 갑자기 사용 가능해진다. 어떻게?

C#에는 확장 메서드(extension method)라는 기능이 있다. 다른 클래스에 밖에서 메서드를 “덧붙이는” 기법.

```
1 namespace System.Linq
2 {
3     public static class Enumerable
4     {
5         public static int Sum(this IEnumerable<int> source) { ... }
6     }
7 }
```

핵심은 첫 매개변수의 `this` 키워드. 이 표시 덕분에 `scores.Sum()`이라고 쓰면 마치 `List<int>`의 메서드인 것처럼 호출된다.

이는 `using`으로 “보이게”만 한 것이지, `List<int>` 자체는 손대지 않았다. `using System.Linq;`를 빼면 `Sum`, `Where` 등이 보이지 않는다.

실험해 보기

- `scores.OrderByDescending(x => x).Take(3)`으로 상위 3개만 출력해 보자.
- `scores.Select(x => x * 2)`는 무엇을 만들까? 그 결과를 `.ToList()`로 새 List로 변환해 보자.
- `List<int>`을 `List<string>`으로 바꾸고 (`Add("apple")`, `Add("banana")` 등) `.OrderBy(s => s.Length)`로 글자 수 순서로 정렬해 보자. 같은 LINQ 메서드가 어떤 타입에든 통한다는 점이 핵심.

8. Lab 6 □ Coroutine과 람다

3. 람다의 다양한 모양

8.1. 문제

“매 1초마다 무언가를 한다”라는 시간이 흐르는 절차를 코드로 표현하고 싶다. `Update`에서 타이머를 직접 굴리는 방식은 동작은 하지만 의도가 흐려진다.

8.2. 콘솔이라면 비슷하게

콘솔 C#에서도 `IEnumerable`와 `yield return`을 통해 “값을 하나씩 돌려주는 함수”를 만들 수 있다.

```

1 static IEnumerable<int> CountUp(int start)
2 {
3     int n = start;
4     while (true)
5     {
6         yield return n;    // pause and return n; resume next time
7         n++;
8     }
9 }
10
11 static void Main()
12 {
13     foreach (int v in CountUp(1).Take(5))
14         Console.WriteLine(v);    // 1 2 3 4 5
15 }

```

Unity의 Coroutine은 이 `yield` 메커니즘을 “매 프레임 시간 흐름”에 적용한 것이다.

8.3. Unity 데모

C 키를 누르면 Tick 코루틴을 시작/중지. 1초마다 카운트가 올라가고, 콜백으로 그 값을 받아 출력한다.

```

1 using System;
2 using System.Collections;
3 using UnityEngine;
4
5 public class Lab6_Coroutine : MonoBehaviour
6 {
7     [SerializeField] private float interval = 1f;
8
9     private Coroutine running;

```



```

10
11 void Update()
12 {
13     if (Input.GetKeyDown(KeyCode.C))
14     {
15         if (running == null)
16         {
17             // Lambda: pass a function as an argument
18             running = StartCoroutine(Tick(interval, n => Debug.Log($"Tick #{n}")));
19             Debug.Log("Started.");
20         }
21         else
22         {
23             StopCoroutine(running);
24             running = null;
25             Debug.Log("Stopped.");
26         }
27     }
28 }
29
30 IEnumerator Tick(float wait, Action<int> onTick)
31 {
32     int count = 0;
33     while (true)
34     {
35         yield return new WaitForSeconds(wait);
36         count++;
37         onTick(count); // invoke the passed-in function
38     }
39 }
40 }

```

8.4. Unity 구현 가이드

1. Lab6 GameObject 만들기

Hierarchy → Create Empty → 이름 Lab6.

2. 스크립트 작성 & 부착

Lab6_Coroutine.cs를 작성하고 Lab6에 드래그.

3. Inspector에서 Interval 조정

Inspector에 Interval = 1이 보인다. 0.5로 바꾸면 0.5초마다, 2로 바꾸면 2초마다 카운트 한다. Play 중에도 변경 가능하지만, Play 종료 시점에 원래 값으로 되돌아간다는 점에 주의.

4. 재생

Play → C 키. Console에 Started. 가 출력되고, 1초마다 Tick #1, Tick #2...가 누적된다. 다시 C 키를 누르면 Stopped. 가 찍히고 카운트가 멈춘다.

5. 관찰 포인트

Tick이 도는 동안에도 Unity의 다른 Update는 정상 동작한다. Coroutine은 “양보(yield)”와 “재개”를 반복할 뿐, 멀티스레드가 아니다.

8.5. 여기서 배우는 C# 개념

IEnumerator와 yield return

yield return X를 만나면 메서드의 실행이 중단되고, X가 호출자에게 “돌려진다”. 다음에 호출자가 “계속해 줘”라고 요청하면 중단된 곳부터 다시 실행된다.

Unity의 StartCoroutine은 매 프레임 “계속해 줘”라고 호출하면서, yield return이 돌려준 객체 (예: WaitForSeconds(1f))에 따라 언제 재개할지를 결정한다.

중요: 코루틴은 멀티스레드가 아니다. 단일 스레드에서 “양보(yield)”와 “재개”를 반복할 뿐이다.

람다와 함수 인자

`n => Debug.Log($"Tick #{n}")`는 이름 없는 함수 (람다 식)다.

- `n =>`: “n을 받는다”
- `Debug.Log(...)`: “이걸 하고 void를 돌려준다”

이 람다는 `Action<int>` 타입에 그대로 들어간다. 함수를 값처럼 변수에 담거나, 인자로 넘기는 사고방식이 함수형 프로그래밍의 핵심 한 줄이다.

함수를 인자로 넘기는 “콜백(callback)” 패턴

Tick 메서드의 시그니처를 다시 보자:

```
1 IEnumerator Tick(float wait, Action<int> onTick)
2 {
3     ...
4     onTick(count); // call the function we were given
5 }
```

Tick은 “매 wait초마다 카운트하면서, 카운트 값을 누군가에게 보고한다”. 누구에게 보고하는지는 호출자가 결정한다 — 인자로 넘긴 `onTick`이 그 “누구”다.

이런 “나중에 호출해 줘”라는 의미의 함수 인자를 콜백(callback)이라 부른다. `Action<T>`, `Func<T, R>` 모두 콜백을 표현하는 타입이다.

콜백을 분리하면 무엇이 좋은가? Tick 안에 `Debug.Log($"Tick {count}")`라고 박혀 있으면, “화면 텍스트 갱신”이나 “사운드 재생”으로 바꾸려면 Tick 자체를 수정해야 한다.

콜백으로 넘기면 Tick은 그대로, “무엇을 할지”만 호출자가 람다로 정한다:

```
1 StartCoroutine(Tick(1f, n => Debug.Log($"Tick {n}"))); // console
2 StartCoroutine(Tick(1f, n => labelText.text = $"#{n}")); // UI
3 StartCoroutine(Tick(1f, n => audio.Play())); // sound
```

Lab 4의 event와 비슷한 “책임 분리” 사상이 콜백에도 흐른다.

yield return이 돌려주는 “객체”의 종류

Unity Coroutine은 yield return이 무엇을 돌려주느냐에 따라 다음에 언제 재개할지를 결정한다. 자주 쓰이는 모양들:

- `yield return null;` — 다음 프레임에 재개. 가장 자주 씀.
- `yield return new WaitForSeconds(t);` — t초 후 재개. 게임 시간(Time.timeScale 영향 받음).

- `yield return new WaitForSecondsRealtime(t);` — 실제 시간 `t`초 후 재개. `timeScale` 영향 안 받음.
- `yield return new WaitUntil(() => 조건);` — 조건이 `true`가 될 때까지 대기.
- `yield return new WaitForEndOfFrame();` — 그 프레임의 모든 렌더링이 끝난 후 재개.

`yield return`이 마치 “편지”라면, 우리가 적은 내용에 따라 Unity가 “언제 답장 줄지”를 결정하는 셈이다.

실험해 보기

- `interval`을 `0.5f`로 줄이면 더 빨리 카운트한다. `Time.timeScale = 0.1f`로도 바꿔 보자.
- Tick을 호출할 때 콜백을 두 가지로 분기해 보자: 짝수일 때만 `Debug.LogWarning`, 홀수일 때 `Debug.Log`. 람다 안에 `if`를 넣어도 좋다.
- `yield return new WaitForSeconds(wait)`을 `yield return null`로 바꿔 보자. 매 프레임마다 콜백이 호출되어 “카운트가 미친 듯이 빠르게” 올라간다. “`yield return`이 무엇을 돌려주느냐에 따라 재개 시점이 결정된다”는 원리를 체감.

9. 이번 강의에서 배운 C# 개념 정리

9.1. 한눈에 보기

C# 기능	Lab
<code>enum + switch</code> 식	Lab 1 (큐브 색)
프로퍼티, <code>=></code>	Lab 2 (HP)
<code>interface</code>	Lab 3 (도형)
<code>event / Action<T></code>	Lab 4 (Counter)
제네릭 + LINQ + 람다	Lab 5 (점수)
<code>Coroutine</code> + 람다	Lab 6 (Tick)

9.2. Unity는 우리에게 무엇을 해 주었는가?

이번 강의의 모든 Lab은 콘솔 C#으로도 만들 수 있었다. 하지만 Unity 위에서 동작시키니 다음이 좋아졌다.

- `enum`이 큐브 색으로 바뀌어 보인다.
- `event`가 소리/색/위치로 동시에 반응하는 것을 눈으로 본다.
- `Coroutine`이 시간을 두고 화면에서 펼쳐지는 것을 본다.
- 키보드 입력으로 지금 당장 실험할 수 있다.

C#은 본질이고, Unity는 그 본질을 더 잘 보여 주는 무대다. 이 강의의 핵심 자세다.

10. 단계별 도전 과제

10.1. Phase 1 □ 같은 Lab에 기능 한 줄 더

과제 1.1

Lab 1에 `Mood.Excited`를 추가하고, 키 4를 눌렀을 때 큐브가 무지개색으로 천천히 변하게 하라. 힌트: `Color.HSVToRGB(t, 1, 1), Time.time`.

과제 1.2

Lab 2에 HpBar UI(슬라이더)를 추가하고, HP가 변할 때 슬라이더가 따라 움직이게 하라. Lab 4의 이벤트 패턴을 적용해, `Lab2_Property`에 `event Action<int> OnHpChanged`를 추가하고 HpBar가 그것을 구독하게 한다. Lab 2와 Lab 4를 결합하는 연습이다.

10.2. Phase 2 □ 두 Lab을 합치기

과제 2.1: “작은 인벤토리”

`IItem` 인터페이스(`string Name; int Value;`)와 `List<IItem>` 인벤토리를 만든다. `Sword`, `Potion`, `Key` 등 여러 구현 클래스를 만들고, LINQ로 “가장 비싼 아이템”, “Potion 종류만 필터링” 같은 질의를 해 보자.

Lab 3 + Lab 5의 결합. 아이템 추가 시에도 인벤토리 코드는 한 줄도 바뀌지 않아야 한다.

과제 2.2: “상태 기계”

`enum CharacterState { Idle, Walking, Running, Jumping }`로 캐릭터 상태를 정의하고, 상태가 바뀔 때마다 `event Action<CharacterState> OnStateChanged`를 발생시킨다. 큐브 색이 상태 별로 바뀌는 `StateColorView`와, 상태 이름을 화면에 출력하는 `StateLabelView`를 별도로 작성. Lab 1 + Lab 4의 결합.

10.3. Phase 3 □ 자유 실험

과제 3: “내가 가장 헛갈리는 C# 기능 한 가지를 Lab으로”

이 강의의 6개 Lab은 시작점일 뿐이다. 자신이 잘 이해되지 않는 C# 기능 하나를 골라 스스로 Lab 하나를 설계하고 만들어 보자.

추천 주제:

- `struct` vs `class`: 값 복사 vs 참조 복사 (큐브 두 개로 시연)
- `out/ref` 파라미터
- nullable 타입 `int?`와 ??
- `abstract class` vs `interface`의 차이
- `static` 메서드/필드
- 예외 처리 `try/catch/finally`

문제 정의 → 콘솔 코드 → Unity 데모 → 실험 과제, 이 흐름을 본인이 작성해 본다. “누군가에게 설명하는 것”만큼 자신이 잘 이해했는지 확인하는 빠른 길은 없다.

11. 이번 강의에서 배운 핵심 개념

- 이 강의의 주인공은 C#이다. Unity는 **Console.WriteLine**보다 훨씬 친절한 무대다.
- **enum + switch** 식은 “상태”를 안전하고 명확하게 표현하게 해 준다.
- 프로퍼티는 “외부 접근의 범위”를 한 줄로 조절하는 강력한 도구다.
- 인터페이스는 “공통 능력”만 약속하여, 새로운 종류가 추가되어도 호출 코드를 바꾸지 않게 해 준다.
- **event/Action<T>**는 발신자와 수신자를 이름으로 묶지 않고 연결한다.
- 제네릭과 LINQ는 컬렉션을 타입 안전하게 + 함수형 스타일로 다루게 해 준다.
- **Coroutine**과 람다는 “시간 흐름”과 “함수를 값으로” 다루는 두 가지 강력한 사고방식이다.

다음 시간엔?

이번 강의의 6개 Lab을 응용해, “C# 기능 두세 가지를 한꺼번에 쓰는 작은 프로젝트”로 발전시킨다. 도전 과제 Phase 2가 그 시작점이다. 자신이 만든 Lab을 정리해 두면, 학기말 발표에서 좋은 자료가 된다.